IFM 2.1: Programmieren 2 (Sommer 2025)

> … And, lastly, there's the explosive growth in demand, which has led to many people doing it who aren't any good at it. Code is merely a means to an end. **Programming is an art and code is merely its medium.** Pointing a camera at a subject does not make one a proper photographer. There are a lot of self-described coders out there who couldn't program their way out of a paper bag.
>
> – John Gruber auf daringfireball.net

Read Online:

- GitHub Preview
- Docsify Preview
- PDF-Version

Kursbeschreibung

Sie haben letztes Semester in **Prog1** die *wichtigsten* Elemente und Konzepte der Programmiersprache Java kennen gelernt.

In diesem Modul geht es darum, diese Kenntnisse sowohl auf der Java- als auch auf der Methoden-Seite so zu erweitern, dass Sie gemeinsam größere Anwendungen erstellen und pflegen können. Sie werden fortgeschrittene Konzepte in Java kennenlernen und sich mit etablierten Methoden in der Softwareentwicklung wie Versionierung von Code, Einhaltung von Coding Conventions, Grundlagen des Softwaretests, Anwendung von Refactoring, Einsatz von Build-Tools und Logging auseinander setzen. Wenn uns dabei ein Entwurfsmuster "über den Weg läuft", werden wir die Gelegenheit nutzen und uns dieses genauer anschauen.

Überblick Modulinhalte

1. Fortgeschrittene Konzepte in Java ("Classic Java")
    - Reguläre Ausdrücke, Annotationen, Reflection
    - Generische Programmierung: Generics
    - Parallele Programmierung: Threads
    - ~~CLI~~, ~~Konfiguration~~, fremde APIs nutzen[1]
    - Graphische Oberflächen mit Swing[2]
2. Fortgeschrittene Konzepte in Java ("FP")

[1] als Teilaufgabe im Praktikum
[2] nur als Wiederholung im Praktikum

    - Default-Methoden, Funktionsinterfaces, Methodenreferenzen, Lambdas, Optional, Stream-API
3. Versionierung mit Git
4. Softwarequalität
    - Testen, Coding Conventions & Smells, Refactoring, Javadoc, Logging
5. Entwurfsmuster
    - ~~Strategy~~, Template-Method, ~~Factory-Method~~, ~~Singleton~~, Observer, Visitor, Command, …
6. Bauen von Software
    - Gradle, Docker, Continuous Integration (GitHub Workflows)

(*durchgestrichene Themen nicht im Sommersemester 2025*)

Team

- Carsten Gips (Sprechstunde nach Vereinbarung)
- BC George (Sprechstunde nach Vereinbarung)
- Tutoren (siehe ILIAS-Mitgliederliste)

Kursformat

W	KW	Datum	Einheit 1	Einheit 2	Einheit 3	Einheit 4	Einheit 5		Praktikum (Blatt)		Flipped Classroom (Quizzes)
1	15	11.4.	Orga								
2	16	18.4.						FEIERTAG			
3	17	25.4	Git1: Intro	Git2: Basics	Funktionsinterfaces & Lambdas	Gradle			B01: Hangman (Swing) (Wiederholung)		Stier-Monster (Git Basics, Lambda, Gradle)
4	18	2.5	Git3: Branches	Git4: Brachingstrategien	Methodenreferenzen	Logging			B02: Git-Quest (Git Basics), Calculator (Lambdas), Gradle		Bahnhofs-Moster (Git Branches, Methodenreferenzen, Logging)
5	19	9.5.	Git5: Remotes	Git6: Workflows	Stream API	Record-Klassen			B03: Git-Quest (Branches), LSF-Contact (Methodenref, Logging)		Uhren-Monster (Git Remotes, Streams, Record)
6	20	16.5.	Station I								
7	21	23.5.	JUnit1: Intro	JUnit2: Basics	Optional	Visitor-Pattern			B04: Stream-API (Streams, Git PR), DevDungeon: Damaged Bridge (Lambdas)		Pudding-Monster (JUnit Basics, Optional, Visitor)
8	22	30.5.	JUnit3: Testfall-Ermittlung	JUnit4: Mocking (Mockito)	Defaultmethoden	Observer-Pattern	Continuous Integration		B05: Cat-Cafe (JUnit Basics, Optional, Visitor), DevDungeon: Torch Riddle (Streams, Optional)		Burg-Monster (ÄK/GW, Observer)
9	23	6.6.	Bad Smells	Coding Rules und Metriken	Refactoring	Javadoc			B06: Cycle Chronicles (ÄK&GW, Mocking)		Kneipen-Monster (Clean-Code) Stier-Monster (Javadoc)
10	24	13.6.	Station II								
11	25	20.6.	RegExp	Template-Method-Pattern	Command-Pattern	Annotationen			B07: Bike-Shop (Smells, Refactoring), DevDungeon: Illusion Riddle (Refactoring), Javadoc		Boss-Monster I (RegExp) Annotationen, Template-Method+Command
12	26	27.6.	Generics1: Klassen & Methoden	Generics2: Bounds & Wildcards	Generics3: Type Erasure	Generics4: Polymorphie	Docker		B08: Lexer (RegExp, Template-Method), DevDungeon: Bridge Guard Riddle (Command Pattern)		Stadtwald-Monster (Generics)
13	27	4.7.	Threads1: Intro	Threads2: Synchronisierung	Threads3: High-Level Konzepte	Reflection			B09: Zoo (Generics)		Boss-Monster II (Threads) Reflection
			Prüfungsphase I	Station III							
			Prüfungsphase II	Station IV							
			Vertiefung Java (Classic Java)								
			Vertiefung Java (FP Java)								
			Git								
			Softwarequalität								
			Pattern								
			Building								

Vorlesung (2 SWS)	Praktikum (2 SWS)
Fr, 08:00 - 09:30 Uhr (online)	G1: Fr, 09:45 - 11:15 Uhr (online, Carsten)
(Carsten: *Flipped Classroom*)	G2: Fr, 11:30 - 13:00 Uhr (online, Carsten)
	G3: Fr, 09:45 - 11:15 Uhr (online, BC)

Vorlesung (2 SWS)	Praktikum (2 SWS)
	G4: Fr, 11:30 - 13:00 Uhr (online, BC)

Online-Sitzungen per Zoom (**Zugangsdaten siehe ILIAS**). Sie *können* hierzu den Raum J101 bzw. J104 (vgl. Stundenplan) nutzen.

Fahrplan

Hier finden Sie einen abonnierbaren Google Kalender mit allen Terminen der Veranstaltung zum Einbinden in Ihre Kalender-App.

Monat	Tag	Vorlesung	VL-Quiz	Praktikum
April	11.	Orga (**Zoom**), FAQ		
	18.	**Feiertag**	**Feiertag**	**Feiertag**
	25.	Einführung Versionierung, Git Basics; Lambda-Ausdrücke; Gradle	Q01	B01
Mai	02.	Git-Branches, Branching-Strategien; Methodenreferenzen; Logging	Q02	B02
	09.	Git-Remotes, Git-Workflows; Stream-API; Record-Klassen	Q03	B03
	16.	**Station I**		
	23.	Einführung Testen, JUnit-Basics; Optional; Visitor-Pattern	Q04	B04
	30.	Testfallermittlung, Mocking; Default-Methoden; Observer-Pattern; Continuous Integration (CI)	Q05	B05
Juni	06.	Code-Smells, Coding-Rules, Refactoring; Javadoc	Q06	B06
	13.	**Station II**		
	20.	RegExp; Template-Method-Pattern, Command-Pattern; Annotationen	Q07	B07
	27.	Generics: Klassen und Methoden, Bounds und Wildcards, Type Erasure, Polymorphie; Docker	Q08	B08
Juli	04.	Intro Threads, Synchronisierung, Highlevel Threadkonzepte; Reflection	Q09	B09
Prüfungsphase tbd I		**Station III**		
Prüfungsphase tbd II		**Station IV**		

Abgabe der Übungsblätter jeweils **bis Fr, 08:00 Uhr** im ILIAS, Vorstellung der Lösung im zugehörigen Praktikum. Bearbeitung der Quizzes jeweils **Sa, 00:00 Uhr (Vorwoche) bis Fr, 08:00 Uhr** im ILIAS.

Prüfungsform, Note und Credits

Parcoursprüfung, 5 ECTS (PO23)

Da Sie das Programmierhandwerk erlernen und üben und vertiefen sollen, dürfen Sie im Rahmen dieser Lehrveranstaltung noch keine KI-gestützten Assistenten benutzen. Lösungen, die dennoch ganz oder teilweise unter Zuhilfenahme von KI-Unterstützung erstellt wurden, werden wie nicht abgegeben behandelt.

Prüfung im ersten Zeitraum

1. **Quizzes**: mind. 5 der 9 Quizzes bestanden (ohne Note/Punkte) (Einzelbearbeitung, fristgerecht bis zur jeweiligen Vorlesung, je Quiz bis zu 3x wiederholbar)
2. **Praktikum**: mind. 5 der 9 Übungsblätter bestanden (ohne Note/Punkte) (bis zu 3er Teams, alle Aufgaben eines Blattes bearbeitet, individuelle(!) fristgerechte Abgabe der Lösungen im ILIAS als aussagekräftiges *Post Mortem*[3], Vorstellung der Lösungen im Praktikum durch jedes Teammitglied, aktive Beteiligung an der Diskussion im Praktikum)
3. **Station I**: ILIAS-Test (30 Minuten in Minden im B40)
4. **Station II**: ILIAS-Test (30 Minuten in Minden im B40)
5. **Station III**: ILIAS-Test (30 Minuten in Minden im B40)

Station I und II finden im Vorlesungsslot statt (Aufteilung siehe separate Ankündigung), Station III im ersten Prüfungszeitraum (Termin wird noch vom Prüfungsamt bekanntgegeben). Die Punkte der beiden besseren Stationen werden summiert für die Berechnung der Note.

Gesamtnote: 4.0: ab 50%, alle 5% nächste Teilnote, 1.0: ab 95% (jeweils nur wenn Quizzes bestanden und Praktikum bestanden)

Bei mind. drei über das Minimum hinaus bestandenen Quizzes und/oder Aufgabenblättern verbessert sich die Gesamtnote um eine Teilnote.

(Hinweise zur Prüfungsvorbereitung für Station I bis III)

Prüfung im zweiten Zeitraum

1. **Station IV**: Schriftliche Prüfung (digitale Klausur) 90 Minuten in Minden im B40

Gesamtnote: 4.0: ab 50%, alle 5% nächste Teilnote, 1.0: ab 95%

(Hinweise zur Prüfungsvorbereitung für Station IV)

[3] **Post Mortem**: Jede Person beschreibt in der ILIAS-Abgabe individuell(!) die Bearbeitung des jeweiligen Aufgabenblattes zurückblickend mit ca. 200 bis 400 Wörtern. Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein: (a) Zusammenfassung: Was wurde gemacht? (b) Implementierungsdetails: Kurze Beschreibung besonders interessanter Aspekte der Umsetzung. (c) Was war der schwierigste Teil bei der Bearbeitung? Wie haben Sie dieses Problem gelöst? (d) Was haben Sie gelernt oder (besser) verstanden? (e) Team: Mit wem haben Sie zusammengearbeitet? (f) Link zum Repo mit der Lösung (erst ab Blatt 04).

Materialien

Literatur

1. "**Java ist auch eine Insel**". Ullenboom, C., Rheinwerk-Verlag, 2021. ISBN 978-3-8362-8745-6.
2. "**Pro Git** (Second Edition)". Chacon, S. und Straub, B., Apress, 2014. ISBN 978-1-4842-0077-3.
3. "The Java Tutorials". Oracle Corporation, 2024.
4. "Learn Java". Oracle Corporation, 2025.

Tools

- JDK: **Java SE 21 (LTS)** (Oracle oder Alternativen, bitte 64-bit Version nutzen)
- IDE: Eclipse IDE for Java Developers oder IntelliJ IDEA (Community Edition) oder Visual Studio Code oder Vim oder …
- Git

Förderungen und Kooperationen

Förderung durch DH.NRW (Digi Fellowships)

Die Überarbeitung dieser Lehrveranstaltung wurde vom Ministerium für Kultur und Wissenschaft (MKW) in NRW im Einvernehmen mit der Digitalen Hochschule NRW (DH.NRW) gefördert: "Fellowships für Innovationen in der digitalen Hochschulbildung" (*Digi Fellowships*).

Kooperation mit dem DigikoS-Projekt

Diese Vorlesung wurde vom Projekt "Digitalbaukasten für kompetenzorientiertes Selbststudium" (*DigikoS*) unterstützt. Ein vom DigikoS-Projekt ausgebildeter Digital Learning Scout hat insbesondere die Koordination der digitalen Gruppenarbeiten, des Peer-Feedbacks und der Postersessions in ILIAS technisch und inhaltlich begleitet. DigikoS wird als Verbundprojekt von der Stiftung Innovation in der Hochschullehre gefördert.

LICENSE

Unless otherwise noted, this work by Carsten Gips and contributors is licensed under CC BY-SA 4.0. See the credits for a detailed list of contributing projects.

> **Last modified:** 95cae66 (Readme: fix typo, 2025-04-11)